

Лабораторні роботи 1,2

Створення БД в MongoDB

1.1 Мета роботи: створити БД з використанням документоорієнтованої СУБД MongoDB

1.2 Порядок виконання роботи

1.2.1 Встановити MongoDB Community Edition

Завантажте MongoDB з офіційного сайту:

<https://www.mongodb.com/try/download/community>

Виберіть вашу операційну систему (Windows, macOS або Linux). Завантажте останню версію для наявної у Вас операційної системи (Windows, macOS або Linux).

Для ОС Windows:

- Запустіть завантажений MSI-файл
- Дотримуйтесь інструкцій майстра встановлення
- Рекомендується встановити MongoDB як службу (MongoDB as a Service)

1.2.2 Для Windows MongoDB буде запущено як службу автоматично після встановлення. Також можна запустити вручну через командний рядок:

```
C:\Program Files\MongoDB\Server\6.0\bin\mongod.exe --dbpath="c:\data\db"
```

1.2.3 Для перевірки роботи MongoDB слід підключитися до MongoDB Shell:

```
mongo
```

Виконайте команду для перевірки доступних баз даних:

```
show dbs
```

Також для розгортання MongoDB можна використовувати Docker (в такому випадку слід запустити контейнер MongoDB, а потім підключитися до контейнера

1.2.4 Проектування структури зберігання даних

Розглянемо предметну область: зберігання заміток користувачів. Для цього створимо колекції Users (для зберігання даних про користувачів) та Notes (для зберігання інформації про замітки користувачів)

```
// Підключення до MongoDB
```

```
const db = connect('mongodb://localhost:27017/notes_db');
```

```

// Створення колекції Users
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["UserName", "Password"],
      properties: {
        UserName: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        Password: {
          bsonType: "string",
          description: "must be a string and is required"
        }
      }
    }
  }
});

// Створення індексів для колекції Users
db.users.createIndex({ "Id": 1 }, { unique: true });
db.users.createIndex({ "UserName": 1 }, { unique: true });

// Створення колекції Notes
db.createCollection("notes", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["Title", "Text", "UserId"],
      properties: {
        Title: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        Text: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        UserId: {
          bsonType: "binData",
          description: "must be a UUID and is required"
        },
        CreatedAt: {
          bsonType: "date",
          description: "date when note was created"
        },
        UpdatedAt: {
          bsonType: "date",
          description: "date when note was last updated"
        }
      }
    }
  }
});

```

```
});

// Створення індексів для колекції Notes
db.notes.createIndex({ "Id": 1 }, { unique: true });
db.notes.createIndex({ "UserId": 1 });

print("Колекції 'users' та 'notes' успішно створені з індексами.");
```

Щоб виконати скрипт для створення колекцій слід зберегти скрипт у файл (наприклад, create_collections.js), адалі його виконати через MongoDB Shell:

mongo create_collections.js

1.2.5 Додамо інформацію про користувачів та замітки:

```
// Підключення до бази даних
use notes_db

// Додавання першого користувача з відомим ідентифікатором
db.users.insertOne({
  "Id": UUID("11111111-1111-1111-1111-111111111111"),
  "UserName": "user1",
  "Password": "password1"
})

// Додавання другого користувача з автоматичною генерацією id
db.users.insertOne({
  "UserName": "user2",
  "Password": "password2"
})

// Додавання третього користувача
db.users.insertOne({
  "Id": UUID("33333333-3333-3333-3333-333333333333"),
  "UserName": "user3",
  "Password": "password3"
})
```

Щоб дізнатися згенеровані ID для подальшого використання:

```
// Отримання списку користувачів з їх ID
const users = db.users.find({}, {UserName: 1}).toArray()
users.forEach(user => {
  print(`Користувач: ${user.UserName}, ID: ${user._id}`)
})
```

Додамо замітки для першого користувача:

```
db.notes.insertOne({
  "Id": UUID("aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa"),
```

```

    "Title": "Лабораторна робота 1",
    "Text": "Зробити лб1",
    "UserId": UUID("11111111-1111-1111-1111-111111111111"),
    "CreatedAt": new Date(),
    "UpdatedAt": new Date()
  })

  // Друга замітка для user1
  db.notes.insertOne({
    "Id": UUID("bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbbbb"),
    "Title": "Реферат",
    "Text": "Підготувати реферат до 01.04.2025",
    "UserId": UUID("11111111-1111-1111-1111-111111111111"),
    "CreatedAt": new Date(),
    "UpdatedAt": new Date()
  })

  // Перша замітка для user3 (позначена як "Виконана")
  db.notes.insertOne({
    "Id": UUID("ddddddddd-dddd-dddd-dddd-ddddddddddddd"),
    "Title": "NoSQL. Лабораторна робота №3",
    "Text": "Зробити лабораторну роботу",
    "UserId": UUID("33333333-3333-3333-3333-333333333333"),
    "Status": "Виконана",
    "CreatedAt": new Date(),
    "UpdatedAt": new Date()
  })

  // Перша замітка для user2 (попередньо необхідно дізнатися _id)
  const userId = db.users.findOne({UserName: "user2"})._id

  // Додаємо першу замітку для цього користувача
  db.notes.insertOne({
    "Title": "Підготувати V&S",
    "Text": "Для ККП написати функціональні вимоги і оформити у вигляді V&S",
    "UserId": userId, // Використовуємо отриманий _id
    "CreatedAt": new Date(),
    "UpdatedAt": new Date()
  })

```

Виконаємо перевірку введених даних:

```

// Знаходимо всі замітки для нашого користувача
const userNotes = db.notes.find({"UserId": userId}).toArray()
printjson(userNotes)

```

1.2.6 Редагування даних:

Необхідно для всіх заміток, в тексті або в назві яких є "лабораторна робота" встановити статус "виконано":

```

// Один запит для обох полів

```

```

db.notes.updateMany(
  {
    $or: [
      { "Title": /лабораторна робота/i },
      { "Text": /лабораторна робота/i }
    ]
  },
  {
    $set: { "Status": "виконано", "UpdatedAt": new Date() }
  }
)

// Знаходимо всі замітки для нашого користувача

```

Перевіримо результати

```

// Знайти всі замітки зі статусом "виконано"
db.notes.find({ "Status": "виконано" }).pretty()

// Або знайти саме ті, що містять "лабораторна робота"
db.notes.find({
  $or: [
    { "Title": /лабораторна робота/i },
    { "Text": /лабораторна робота/i }
  ]
}).pretty()

```

Необхідно додати текст "Терміново виконати" до наявного тексту для заміток користувача з ім'ям User2

```

db.notes.updateMany(
  {
    "UserId": user2Id
  },
  {
    $set: {
      "Text": { $concat: ["$Text", " Терміново виконати!"] },
      "UpdatedAt": new Date()
    }
  }
)

```

Якщо поле Text може бути null, слід додати перевірку:

```

$set: {
  "Text": {
    $ifNull: [
      { $concat: ["Терміново виконати! ", "$Text"] },
      "Терміново виконати!"
    ]
  }
}

```

Перевіримо результати

```
db.notes.find({ "UserId": user2Id }).pretty()
```

1.2.7 Обчислення результатів

Визначити кількість заміток у кожного користувача (можна використовувати різні варіанти, приклад одного з них):

```
db.users.aggregate([
  {
    $lookup: {
      from: "notes",
      localField: "Id", // або "_id", залежно від вашої схеми
      foreignField: "UserId",
      as: "user_notes"
    }
  },
  {
    $project: {
      "UserName": 1,
      "notes_count": { $size: "$user_notes" }
    }
  }
])
```

Якщо відомий користувач, для якого слід визначити кількість заміток, можна використати такий підрахунок:

```
// Отримуємо ID користувача
const userId = db.users.findOne({ "UserName": "User2" }).Id;

// Рахуємо кількість заміток
const count = db.notes.countDocuments({ "UserId": userId });
print(`Користувач User2 має ${count} заміток`);
```